

DROIDBRIDGE

ADB-Powered Android Device Management Tool



Project Requirements, Feature Specification & Implementation Record

Version 1.0.3

Released June 2026

Proprietary Source-Available License

Table of Contents

1. Project Overview	6
2. Goals & Design Principles	6
Primary Goals	6
Design Principles	6
3. Technology Stack	7
3a. Platform Strategy	7
Single Codebase, Per-Platform Releases	7
Why Not Separate Codebases	8
Development Order by Platform	8
4. Module 1 — Device Manager	8
1.1 Device Detection & Connection	8
1.2 Device Information Display	9
1.3 Storage Overview	9
1.4 Connection Health	9
5. Module 2 — File Browser	9
2.1 Directory Navigation	9
2.2 File Listing & Filtering	10
2.3 File Preview	10
2.4 File Operations (via right-click menu)	10
6. Module 3 — Smart Transfer Engine	10
3.1 Transfer Modes	10
3.2 Performance	10
3.3 Resume & Reliability	11
3.4 Conflict Resolution	11
3.5 Post-Transfer Verification	11

7. Module 4 — WhatsApp Toolkit	11
4.1 Media Analysis & Reporting	12
4.2 Folder Coverage	12
4.3 Backup Features	13
4.4 Backup Organization (Optional)	13
4.5 File Fixing & Cleanup	14
4.6 Cleanup & Deletion	15
4.7 Pre/Post Cutoff Comparison	15
4.8 Database & System Files Backup	15
4.9 Restore (Media)	15
4.10 Status Saver	16
8. Module 5 — Storage Analyzer	16
5.1 Full Storage Breakdown	16
5.2 App Storage Analysis	16
5.3 Media Analysis	17
5.4 Large File Finder	17
5.5 Cleanup Suggestions	17
9. Module 6 — Backup Manager	17
6.1 Backup Profiles	17
6.2 Backup Execution	17
6.3 Backup Verification	18
6.4 Backup History	18
6.5 Restore	18
6.6 Contacts & Call Log Export (GUI, planned for sub-phase 6.4)	18
10. Module 7 — Search & Discovery	19
7.1 Search Capabilities	19
7.2 Search Results	19
7.3 Quick Discovery Presets	19
11. Module 8 — App Manager	19

8.1 App Listing	19
8.2 Cache Management	20
8.3 App Uninstall	20
8.4 APK Extraction	20
8.5 Bloatware Manager	20
12. Module 9 — Reports & Analysis	20
9.1 WhatsApp Analysis Reports	20
9.2 Storage Reports	21
9.3 Backup Reports	21
9.4 Deletion Reports	21
9.5 Report Formats	21
9.6 Session Logs	21
13. CLI Interface Specification	22
Command Structure	22
Key Commands	22
14. GUI Interface Specification	23
Layout	23
Key UI Requirements	23
15. Non-Functional Requirements	24
16. Implementation Phases (1–6)	24
Phase 1 — Foundation: ADB Core & Device Manager	25
Phase 2 — File Operations: Browser, Search, Transfer	25
Phase 3 — WhatsApp Toolkit: Analysis, Backup, Cleanup	26
3.1 Analysis Sub-Phase	26
3.2 Backup Sub-Phase	26
3.3 Organization Sub-Phase (Optional Step)	26
3.4 Cleanup Sub-Phase	27
3.5 Database Sub-Phase	27
Phase 4 — Storage, Apps & Backup Manager + Cross-Platform	27

Phase 5 — Reports & Analysis Module	28
Phase 6 — GUI & Packaging Complete (v1.0.0)	28
17. Future Ideas / Research Backlog	29
17.1 WhatsApp Cross-Device "Cloner" (like paid transfer tools)	29
19. Version 1.0.0 — Implementation Notes & Deviations	30
19.1 Phase Completion Summary	30
19.2 ADB Binary	30
19.3 GUI Sub-Phases (Phase 6 breakdown)	31
19.4 Features Added Beyond Spec	31
19.5 Features Deferred from Spec	32
19.6 Known Limitations (non-bugs)	32
20. Deferred, Blocked & Future Work	32
Priority future work (post v1.0.0)	33
21. Architecture Overview	33
21.1 Shared Core	33
21.2 CLI Architecture	34
21.3 GUI Architecture	34
21.4 Layered Component Map	35
22. About the Developer	35

1. Project Overview

DroidBridge is an open-source, cross-platform desktop tool for managing Android devices via ADB (Android Debug Bridge). It provides significantly faster file transfers than standard MTP, intelligent media analysis and organization, a complete WhatsApp backup and cleanup toolkit, app management, and rich report generation — all without requiring internet access or cloud services.

The tool was born from a real need: Android users face painfully slow MTP transfers, no smart file discovery, and no native tools for managing large WhatsApp media archives accumulated over years. DroidBridge solves all of these problems in a single, privacy-first, offline application.

Core Value Proposition: 3–5x faster than MTP | Smart media analysis | Complete WhatsApp toolkit | 100% offline & private

2. Goals & Design Principles

Primary Goals

- Replace slow MTP with ADB-powered transfers that are 3–5x faster
- Provide complete WhatsApp media analysis, backup, organization, and cleanup
- Enable intelligent storage analysis and space recovery on Android devices
- Work entirely offline with no cloud dependency or internet requirement
- Be accessible via both CLI (for power users) and GUI (for general users)
- Generate detailed human-readable reports for all operations

Design Principles

Principle	Description
Safety First	No destructive operation runs without explicit confirmation. Every deletion shows a preview first.
Verified Backup	All backups are verified with file count and size checks before any phone deletion.
Resume Support	All transfers and operations can be interrupted and resumed without starting over.
Offline & Private	No internet required. No telemetry. No cloud. All data stays local.
Cross-Platform	Works on Windows, Linux, and macOS with the same feature set.
ADB Bundled	ADB is bundled with the tool — no separate Android SDK installation required.

Principle	Description
Organized Output	Backups can be organized into intuitive folder structures (optional, see 4.4).
Detailed Logging	Every session is logged with timestamps, counts, sizes, and error reports.

3. Technology Stack

Component	Technology	Reason
Core Language	Python 3.10+	Cross-platform, rich ecosystem, fast prototyping
CLI Interface	Click / argparse	Clean argument parsing, help generation
GUI Framework	PyQt6 or Tkinter	Cross-platform desktop UI, no browser required
ADB Communication	subprocess + adb binary	Direct ADB commands, no Java dependency
ADB Binary	Bundled platform-tools	No user installation required
File Analysis	os, pathlib, re, mimetypes	Built-in, no dependencies
Report Generation	Jinja2 + HTML/PDF	Rich formatted reports
Data Storage	SQLite	Local session and backup metadata
Packaging	PyInstaller	Single executable per platform
Version Control	Git + GitHub	Source control and collaboration

Note: The tool bundles ADB platform-tools for Windows, Linux, and macOS. Users do not need to install Android SDK separately.

3a. Platform Strategy

DroidBridge is built as a **single Python codebase** shared across all platforms — not separate codebases per OS. Platform-specific differences (ADB binary, sleep inhibitor, paths) are isolated behind a small platform abstraction layer.

Single Codebase, Per-Platform Releases

- One Python codebase for core logic, CLI, and GUI
- Platform-specific code isolated in `core/platform/` (`windows.py` , `linux.py` , `macos.py`)

- Bundled ADB binaries: `platform-tools-windows`, `platform-tools-linux`, `platform-tools-darwin`
- Sleep inhibitor implementations per OS:
- Linux: `systemd-inhibit`
- Windows: `SetThreadExecutionState` (ctypes)
- macOS: `caffeinate` subprocess
- Build & release separately per platform via PyInstaller (one executable per OS), but from the same source

Why Not Separate Codebases

- WhatsApp toolkit logic (the core value) is 100% identical across platforms
- Maintaining 3 codebases triples bug-fix effort for marginal gain
- ADB commands themselves are identical across platforms — only the binary path and OS-level helpers differ

Development Order by Platform

Stage	Primary Platform	Reason
Phase 1–3 (CLI core)	Linux (Ubuntu)	Development environment, fastest iteration, matches original sessions
Phase 4 (Polish)	Linux + Windows	Add Windows ADB path handling and sleep inhibitor
Phase 5/6 (GUI)	Linux + Windows + macOS	GUI framework (PyQt6) is cross-platform from day one
Release	All three	PyInstaller builds produced for Windows, Linux, macOS from same tag

Recommendation: Build and validate on Linux first (matches the original development environment), then add Windows/macOS platform adapters in Phase 4. The CLI works on all 3 platforms before GUI work begins.

4. Module 1 — Device Manager

Handles device detection, connection management, and device information display.

1.1 Device Detection & Connection

- Auto-detect connected Android devices via ADB on startup

- Display connection status with clear indicators (connected / unauthorized / offline)
- Guide user through USB debugging setup if not enabled
- Support multiple simultaneously connected devices with device selector
- Reconnect automatically if device disconnects and reconnects
- Show device serial number, model name, Android version

1.2 Device Information Display

- Device model, manufacturer, Android version, build number
- Storage breakdown: total, used, free (internal storage)
- Battery level and charging status
- USB connection type (USB 2.0 / 3.0 / 3.1)
- Estimated transfer speed based on connection type

1.3 Storage Overview

- Visual storage bar showing used vs free
- Breakdown by category: Apps, Images, Videos, Audio, Documents, System
- Top space-consuming apps list
- WhatsApp-specific storage summary

1.4 Connection Health

- ADB daemon status check
- Auto-restart ADB daemon if needed
- USB mode detection and guidance (must be File Transfer / MTP)
- Sleep prevention during active operations

5. Module 2 — File Browser

Browse, search, and preview files on the connected Android device without copying them first.

2.1 Directory Navigation

- Tree-view navigation of phone filesystem
- Show file count and total size per folder
- Hidden file toggle (show/hide `.nomedia`, etc.)
- Breadcrumb navigation path
- Bookmarks for frequently accessed folders
- Quick-jump to common locations (WhatsApp, DCIM, Downloads, etc.)

2.2 File Listing & Filtering

- Sort by: name, date modified, size, file type
- Filter by: file extension, date range, size range
- Show: filename, size, date modified, extension
- Multi-select files and folders
- Select all / deselect all
- Invert selection

2.3 File Preview

- Inline image preview (JPEG, PNG, HEIC, GIF, WebP)
- Video thumbnail preview
- Audio file metadata (duration, bitrate)
- Document type icon with file size
- Preview without copying file to laptop

2.4 File Operations (via right-click menu)

- Copy to laptop (single or batch)
- Delete from phone (with confirmation)
- Rename file on phone
- Show full file path
- Copy file path to clipboard

6. Module 3 — Smart Transfer Engine

ADB-powered file transfer that is 3–5x faster than standard MTP, with resume support, progress tracking, and conflict resolution.

3.1 Transfer Modes

- Phone → Laptop (pull)
- Laptop → Phone (push)
- Batch transfer: multiple files or entire folders
- Selective transfer: transfer only files matching filter criteria
- Mirror mode: sync phone folder to laptop folder

3.2 Performance

- ADB batch commands (500 files per ADB call) for 10–20x speed vs one-by-one

- Automatic USB sleep prevention during transfer
- Transfer speed display (MB/s)
- Estimated time remaining
- Progress bar with file count and bytes transferred

3.3 Resume & Reliability

- Detect interrupted transfers and offer resume
- Skip already-transferred files (by name + size comparison)
- Retry failed transfers automatically (configurable retry count)
- Transfer log saved to disk for post-session review

3.4 Conflict Resolution

- Skip: keep existing file, don't overwrite
- Overwrite: replace existing file
- Rename: add suffix to new file (`_dup` , `_1` , `_2` , etc.)
- Ask each time: prompt user per conflict
- Remember choice for session

Resume takes precedence over conflict mode: a destination file is only a *conflict* (subject to skip/overwrite/rename above) if it has the **same name but a different size** than the source — a genuine naming collision between two different files. If a destination file already exists with the **same name and the same size** as the source, it's treated as "already transferred" (resume support, §3.3) and is left alone regardless of the selected conflict mode. This applies symmetrically to both transfer directions — pull (destination on the laptop) and push (destination on the device) — and has been verified on a real device for all three conflict modes in both directions.

3.5 Post-Transfer Verification

- File count verification: source vs destination
- Total size verification
- Generate transfer report with success/failure summary

7. Module 4 — WhatsApp Toolkit

A comprehensive toolkit for WhatsApp media management built from real-world experience. Covers analysis, backup, organization, cleanup, and database preservation.

Important: This module supports both WhatsApp (`com.whatsapp`) and WhatsApp Business (`com.whatsapp.w4b`) — selectable at runtime.

4.1 Media Analysis & Reporting

- Scan all WhatsApp media folders on the phone
- Detect WhatsApp media path automatically (varies by Android version):
 - `/sdcard/Android/media/com.whatsapp/WhatsApp/Media/` (Android 10+)
 - `/sdcard/WhatsApp/Media/` (older versions)
- Generate report broken down by:
 - Folder type (Voice Notes, Images, Video, Documents, Stickers, Audio, Gifs, etc.)
 - Section (Received, Sent, Private)
 - Year and month
 - File type / extension
 - File count and actual size per group
 - Pre vs post user-defined cutoff date comparison
- Identify orphaned media: files with no linked chat in database — **out of scope on non-rooted devices** (caveat, not a command): `Databases/msgstore.db*` is always `.crypt14`-encrypted, and its decryption key lives in `/data/data/com.whatsapp/files/key`, inside app-private storage that `adb` cannot read without root. Document this caveat rather than implementing orphaned-media detection.
- Detect files with missing or incorrect extensions using MIME type detection
- Detect files with malformed dates (student IDs, serial numbers mistaken as dates)
- Save analysis reports to disk (TXT and HTML formats)

4.2 Folder Coverage

Folder	Content	Notes
WhatsApp Voice Notes	Voice messages (PTT)	Organized by numbered subfolders on newer Android
WhatsApp Images	Received/Sent/Private images	JPEG, PNG, HEIC, WebP
WhatsApp Video	Received/Sent/Private videos	MP4, MOV, MKV, AVI
WhatsApp Audio	Audio files shared in chats	MP3, AAC, M4A, OGG

Folder	Content	Notes
WhatsApp Documents	All document types	PDF, DOCX, PPTX, ZIP, RAR, source code, etc.
WhatsApp Stickers	Received stickers	Backed up — NOT included in Google Drive backup
WhatsApp Sticker Packs	Installed sticker packs	From official WhatsApp store
WhatsApp Backup Excluded Stickers	Third-party sticker packs	Excluded from all backups — manual only
WhatsApp Animated Gifs	GIF files	Received in chats
WhatsApp Profile Photos	Contact profile pictures	Cached profile photos
WhatsApp Video Notes	Short video messages	Circular video bubbles
WallPaper	Chat wallpapers	Custom wallpapers set by user
Databases	msgstore.db and others	Active message database — critical
Backups	Local encrypted backups	.crypt12 / .crypt15 files

4.3 Backup Features

- One-click full WhatsApp backup to selected destination
- Selective backup by media type (e.g. only images + voice notes)
- Direct-to-external-drive backup (bypasses laptop storage)
- Resume interrupted backups using rsync-style logic
- Post-backup verification: file count and size match check
- Backup report saved with timestamp, counts, and sizes
- Backup session logs: all scripts, reports, and summaries preserved
- Prevent laptop sleep during backup using system inhibit

4.4 Backup Organization (Optional)

After pulling files, the user can **optionally** choose to organize them into clean folder structures. Organization is **NOT automatic** — it is a separate step the user explicitly triggers after backup is complete.

Organization is optional. Raw backup (flat folder structure) is always the default. The user chooses whether to organize after backup is verified.

Organization options:

- Organize now — run organization immediately after backup

- Organize later — keep raw backup, run organization as a separate step anytime
- Skip organization — keep raw flat folder structure permanently
- Custom organization — user defines folder structure rules

When organization is chosen, the following structures are applied:

Voice Notes:

```
WhatsApp_Voice_Notes_Organized/
  2017/  2018/  2019/  ...  2026/
    01-Jan/  02-Feb/  ...  12-Dec/
      PTT-20210615-WA0001.opus
    Unknown/  ← files with no date pattern
```

Images & Video:

```
WhatsApp_Images_Organized/
  Received/
    2020/  01-Jan/  ...  12-Dec/
  Sent/
    2020/  01-Jan/  ...  12-Dec/
  Private/
    2024/  ...
```

Documents:

```
WhatsApp_Documents_Organized/
  Received/
    PDFs/      2021/  01-Jan/  ...
    Archives/   ...
    Images/     ...
    Videos/    ...
    Source_Code/ Original_Files/
    Engineering/ Original_Files/
    Presentations/ ...
    Word_Documents/ ...
    Spreadsheets/ ...
    Executables/ ...
    Other/      ...
  Sent/
    (same structure)
```

4.5 File Fixing & Cleanup

- Detect files with no extension using MIME type fingerprinting
- Automatically rename files with correct extension (e.g. `DOC-20220310-WA0001.` → `.pdf`)
- Fix double-dot filenames (e.g. `Sheet..pdf` → `Sheet.pdf`)
- Detect and fix files placed in wrong year folders due to numbers in filename

- Move files with original names (no WhatsApp date) to `original_Files` subfolder
- Validate year range (2009–2030) to reject false date matches
- Validate month range (01–12) to reject false date matches

4.6 Cleanup & Deletion

- Delete media by date cutoff (e.g. delete everything before September 2024)
- Pre-deletion preview showing exact file count and estimated size by year
- Breakdown of what will be deleted vs kept by folder and file type
- Keep/delete by file type (e.g. keep images and audio, delete documents and archives)
- Batch deletion using 500 files per ADB command (10–20x faster than one-by-one)
- Mandatory confirmation step: user must type `'YES DELETE'` to proceed
- Deletion log saved with count of deleted vs failed files
- Post-deletion rescan to verify cleanup was successful
- Resume interrupted deletion: rescan phone and build new delete list from remaining files
- Never delete without verified backup existing first

4.7 Pre/Post Cutoff Comparison

- User defines a cutoff date (default: September 2024)
- Report shows for each folder/type:
 - Pre-cutoff: file count and size
 - Post-cutoff: file count and size
 - No-date files: count and size
 - Total deletable size estimate
- Status tracking: Done / Partial / Pending per month after deletion

4.8 Database & System Files Backup

- Backup `Databases/` folder (msgstore.db, wa.db, axolotl.db, etc.)
- Backup `Backups/` folder (local encrypted .crypt files)
- Backup `accounts/` folder
- Warn user that msgstore.db contains active chats — critical to preserve
- Note encryption status of database files

4.9 Restore (Media)

- Push a previously made WhatsApp media backup (raw or organized) back onto the phone's WhatsApp / WhatsApp Business media path
- Uses Module 3 transfer engine in push mode — same conflict resolution (skip/overwrite/rename) and post-transfer verification as backup

- Media-only: `Databases/ / Backups/ / accounts/` restore is out of scope here — see §6.5 for whole-phone restore and the database-restore caveat in §4.8's Phase 3.5 follow-up

4.10 Status Saver

- Save WhatsApp's currently-active Status images/videos (24h-ephemeral) before they're cleared, for both WhatsApp and WhatsApp Business
- Source: `<base_path>/Media/.Statuses/` — the hidden folder excluded from `scan_media` (§4.1) as internal cache, repurposed here as the status source
- Filters to image/video files only (skips `.nomedia` and other marker files)
- Uses Module 3 transfer engine in pull mode — same conflict resolution and post-transfer verification as backup
- Destination layout: one subfolder per app — `<dest>/WhatsApp/Statuses/` and `<dest>/WhatsApp Business/Statuses/` — avoids filename collisions between the two apps
- GUI (Phase 6): thumbnail grid of current statuses per app (image preview / video thumbnail per §2.3) with checkbox selection, so the user can choose which statuses to save individually rather than saving everything

8. Module 5 — Storage Analyzer

Analyze phone storage in depth to identify what is taking space and find opportunities for cleanup.

5.1 Full Storage Breakdown

- Total storage, used, and free space
- Breakdown by Android category: Apps (installed + data + cache), Images, Videos, Audio, Documents, Downloads, System
- Visual bar chart of storage usage
- Compare against phone total capacity

5.2 App Storage Analysis

- List all installed apps sorted by total size (installed + data + cache)
- Show per-app breakdown: APK size, data size, cache size
- Identify top 20 space consumers
- Flag apps with unusually large cache (potential cleanup targets)
- Distinguish system apps from user-installed apps

5.3 Media Analysis

- Scan all media across entire phone (not just WhatsApp)
- Breakdown by media type: photos, videos, audio, documents
- Identify largest individual files
- Find files older than a specified date
- Find duplicate files by name and size

5.4 Large File Finder

- Find all files above a configurable size threshold (default: 50 MB)
- Sort by size descending
- Show file path, type, and size
- Option to copy or delete found files

5.5 Cleanup Suggestions

- App cache that can be safely cleared
- APK installer files in Downloads
- Thumbnail cache folders
- Old log files and crash reports
- Estimated space recoverable per suggestion

9. Module 6 — Backup Manager

Manage full phone backups with profiles, scheduling, verification, and restore support.

6.1 Backup Profiles

- Create named backup profiles (e.g. 'WhatsApp Only', 'Full Media', 'Documents')
- Define which folders to include/exclude per profile
- Define destination path per profile
- Save and reuse profiles across sessions

6.2 Backup Execution

- Run full or selective backup based on profile
- Direct-to-external-drive support
- Incremental backup: only copy new/changed files since last backup
- Prevent laptop sleep during backup
- Real-time progress with speed, ETA, and file count

6.3 Backup Verification

- Post-backup file count check: source vs destination
- Post-backup size check: source vs destination
- Flag any files that failed to transfer
- Generate verification report
- Block phone-side deletion until backup is verified

6.4 Backup History

- Log each backup with: timestamp, profile used, file count, total size, duration, destination
- Browse backup history in UI
- Compare backups across sessions
- Detect if backup is outdated (last backup > N days ago)

6.5 Restore

- Restore files from backup to phone
- Selective restore: choose specific folders or date ranges
- WhatsApp restore: push organized backup back to correct phone paths
- Conflict resolution during restore (skip/overwrite/rename)

6.6 Contacts & Call Log Export (GUI, planned for sub-phase 6.4)

Decided 2026-06-19, beyond the original file/folder-based scope above — see PROGRESS.md's 2026-06-19 "Decided scope for sub-phase 6.4" entry for the full feasibility analysis.

- Export Contacts (`content query --uri content://contacts/...`) and Call Log (`content://call_log/calls`) via adb, no root required
 - Export formats: vCard/CSV/JSON
 - **Backup-only, one-way** — no restore guarantee (re-inserting contacts via `content insert` risks duplicates; no clean restore path)
 - **Explicitly excluded** (re-raise scope question before adding):
 - **SMS** — read/write access varies too much by OEM/Android version
 - **Accounts** — `dumpsys account` only exposes names/types, never credentials/tokens, so it would be informational-only
 - **Settings** — restoring risks breaking network/UI settings; would need a prominent warning like §4.8's msgstore warning
 - Android's native `adb backup -all` — effectively dead since ~Android 12 (most apps disable it; requires interactive on-device PIN confirmation; can't run headless)
-

10. Module 7 — Search & Discovery

Instantly search and discover files across the entire phone filesystem.

7.1 Search Capabilities

- Search by filename (partial match, wildcard, regex)
- Search by file extension or MIME type
- Search by date range (modified date from filename or filesystem)
- Search by size range (e.g. files between 10 MB and 100 MB)
- Scope search to specific folder or entire phone
- Combine multiple filters in a single search

7.2 Search Results

- Show results with: full path, size, date, extension
- Sort results by any column
- Paginate large result sets
- Select results for batch operations (copy, delete)
- Export search results as CSV or TXT report

7.3 Quick Discovery Presets

- Find all WhatsApp media
- Find all photos (JPEG, PNG, HEIC, RAW)
- Find all videos (MP4, MOV, MKV)
- Find all documents (PDF, DOCX, PPTX, XLSX)
- Find all APK files
- Find files larger than 50 MB
- Find files older than 1 year
- Find files with no extension

11. Module 8 — App Manager

View, manage, and clean up installed apps on the connected Android device.

8.1 App Listing

- List all installed apps (user-installed and system)

- Show per app: name, package ID, version, installed size, data size, cache size
- Filter: user apps only / system apps only / all
- Sort by: name, total size, data size, cache size, install date

8.2 Cache Management

- Clear cache for individual app
- Clear cache for all user apps in one click
- Show estimated space to be freed before clearing
- Exclude specific apps from bulk cache clear

8.3 App Uninstall

- Uninstall user-installed apps via ADB
- Batch uninstall: select multiple apps
- Confirmation dialog before uninstall
- Cannot uninstall system apps (safely blocked)

8.4 APK Extraction

- Extract APK file from any installed app to laptop
- Useful for backing up apps before uninstalling
- Show APK size before extraction

8.5 Bloatware Manager

- Identify pre-installed manufacturer apps (bloatware)
- Disable (not uninstall) bloatware apps safely
- Re-enable disabled apps
- Show which apps are currently disabled

12. Module 9 — Reports & Analysis

Generate rich, detailed reports for all analysis and operations. Save session history for future reference.

9.1 WhatsApp Analysis Reports

- Full media inventory: all folders, all years, all months, file counts, sizes
- Pre vs post cutoff comparison report
- File type breakdown report (extension, count, total size, sorted by size)

- Sent vs Received vs Private breakdown
- Orphaned media report (files with no linked chat) — out of scope on non-rooted devices, see §4.1 caveat
- Documents categorization report (PDFs, Archives, Images, Videos, Code, Engineering, etc.)
- Before/after cleanup comparison

9.2 Storage Reports

- Phone storage breakdown report
- Top apps by size report
- Large files report
- Storage trend: compare reports across sessions to see growth over time

9.3 Backup Reports

- Backup summary: what was backed up, file counts, sizes, duration
- Backup verification report: match status between source and destination
- Backup history log
- Session summary: all operations performed in a session

9.4 Deletion Reports

- Pre-deletion preview report: what will be deleted, broken down by year and type
- Post-deletion report: how many files deleted, how many failed, space freed
- Phone storage before vs after comparison

9.5 Report Formats

Format	Use Case
Plain Text (.txt)	Terminal-friendly, lightweight, human-readable
HTML	Rich formatted report viewable in browser
CSV	Tabular data for import into spreadsheets
JSON	Machine-readable for programmatic processing

9.6 Session Logs

- All scripts generated during a session saved to `session_logs/scripts/`
- Sub-categorized: `analysis/`, `organization/`, `deletion/`
- All reports saved to `session_logs/reports/`
- Session summary updated after each major operation

- Logs automatically synced to backup destination

13. CLI Interface Specification

The CLI provides full access to all features via terminal commands, suitable for scripting and automation.

Command Structure

```
droidbridge <module> <command> [options]
```

Key Commands

Command	Description
<code>droidbridge device info</code>	Show device info and storage breakdown
<code>droidbridge device connect</code>	Check ADB connection and guide setup
<code>droidbridge files browse <path></code>	List files at path on phone
<code>droidbridge files search --type jpg --after 2024-01-01</code>	Search files with filters
<code>droidbridge transfer pull <phone_path> <local_path></code>	Pull files/folders from phone
<code>droidbridge transfer push <local_path> <phone_path></code>	Push files/folders to phone
<code>droidbridge whatsapp scan</code>	Full WhatsApp media scan and report
<code>droidbridge whatsapp analyze --cutoff 2024-09-01</code>	Pre/post cutoff analysis
<code>droidbridge whatsapp backup --dest /media/drive/</code>	Full WhatsApp backup
<code>droidbridge whatsapp backup --type voice_notes,images</code>	Selective backup
<code>droidbridge whatsapp save-status --dest /media/drive/</code>	Save current status images/videos (both apps, per-app subfolders)
<code>droidbridge whatsapp organize --src /backup/ --type images</code>	Organize backup

Command	Description
<code>droidbridge whatsapp delete --before 2024-09-01 --keep images,audio</code>	Delete with preview
<code>droidbridge whatsapp fix-extensions --path /backup/docs/</code>	Fix missing extensions
<code>droidbridge storage analyze</code>	Full phone storage analysis
<code>droidbridge storage apps --top 20</code>	Top apps by size
<code>droidbridge storage large-files --min-size 50mb</code>	Find large files
<code>droidbridge backup run --profile whatsapp_full</code>	Run backup profile
<code>droidbridge backup verify --dest /media/drive/</code>	Verify backup integrity
<code>droidbridge apps list --sort size</code>	List apps by size
<code>droidbridge apps clear-cache --all</code>	Clear all app caches
<code>droidbridge report generate --format html</code>	Generate full HTML report

14. GUI Interface Specification

The GUI provides all features in a clean desktop application suitable for general users.

Layout

- Left sidebar: navigation between modules
- Top bar: device status indicator, connection info, battery
- Main area: module-specific content
- Bottom status bar: current operation progress
- Log panel (collapsible): real-time operation log

Key UI Requirements

- Dark mode and light mode support
- Responsive layout — works at 1280x720 minimum
- All destructive operations require explicit confirmation dialog
- Progress dialogs for all long-running operations with cancel button
- Color-coded status indicators (green=success, red=error, yellow=warning)
- Tooltips on all buttons and options

- Keyboard shortcuts for common actions

15. Non-Functional Requirements

Requirement	Specification
Platforms	Windows 10+, Ubuntu 20.04+, macOS 12+
Android Support	Android 8.0+ (API 26+), tested up to Android 14
Transfer Speed	3–5x faster than MTP (target: 80–150 MB/s on USB 3.0)
Batch Deletion	500 files per ADB command minimum
Scan Performance	500,000+ files scanned in under 5 minutes
Memory Usage	Under 512 MB RAM for normal operations
ADB Dependency	Bundled — no user installation required
Internet Required	None — fully offline
Privacy	No telemetry, no analytics, no cloud
License	Proprietary Source-Available — free for personal use, commercial license required for business use
Language	Python 3.10+ (CLI), PyQt6 or Tkinter (GUI)
Packaging	Single executable via PyInstaller per platform
Documentation	README, CLI help text, in-app tooltips
Error Handling	All ADB errors caught and reported with helpful messages
Logging	All operations logged with timestamps to <code>session_logs/</code>
Safety	No deletion without verified backup + explicit confirmation

16. Implementation Phases (1–6)

The project is broken into 6 phases, each independently usable and testable. Each phase builds on the previous one without requiring rework. The CLI is fully functional after Phase 3; GUI and polish come later.

Each phase ends with a working, testable CLI tool. **Do not start a phase until the previous phase's deliverables pass manual testing on a real device.**

Phase 1 — Foundation: ADB Core & Device Manager

Goal: establish the ADB wrapper and device detection that every other module depends on.

Deliverables:

- Project skeleton: `droidbridge/core`, `modules`, `cli`, `utils`, `reports`
- Bundled ADB binaries for Linux (Windows/macOS binaries added in Phase 4)
- `core/adb.py`: wrapper for `adb devices`, `shell`, `pull`, `push`, with error handling
- Module 1 — Device Manager: detection, info, storage breakdown, connection health
- CLI: `droidbridge device info / connect`
- Basic logging framework (`session_logs/` structure)

Exit Criteria:

- Can detect a connected device and print model, Android version, storage breakdown
- Handles unauthorized/offline device states gracefully with guidance
- ADB daemon restart works if connection drops

Phase 2 — File Operations: Browser, Search, Transfer

Goal: build the generic file operations that the WhatsApp toolkit will reuse.

Deliverables:

- Module 2 — File Browser: directory listing, filtering, sorting
- Module 7 — Search & Discovery: name/type/date/size filters, quick presets
- Module 3 — Smart Transfer Engine:
 - Single-file and folder pull/push
 - Batch ADB commands (500 files per call)
 - Progress tracking (count, bytes, speed, ETA)
 - Conflict resolution (skip/overwrite/rename)
 - Post-transfer verification (count + size)
 - Resume support: detect partial transfers, skip completed files
 - Sleep inhibitor for Linux (`systemd-inhibit`)
- CLI: `droidbridge files browse / search`, `droidbridge transfer pull / push`

Exit Criteria:

- Can pull a 20+ GB folder with progress display and verify file count/size after
- Interrupting and re-running a transfer skips already-copied files

- Search returns correct results for combined filters (type + date + size)
-

Phase 3 — WhatsApp Toolkit: Analysis, Backup, Cleanup

Goal: implement the core value of the project — the full WhatsApp toolkit. This is the largest phase.

3.1 Analysis Sub-Phase

- Auto-detect WhatsApp media path (`com.whatsapp` and `com.whatsapp.w4b`)
- Full scan of all 14 media folder types
- Report by folder/section (Received/Sent/Private)/year/month/extension
- Pre/post cutoff date comparison report
- Orphaned media detection — out of scope on non-rooted devices (see §4.1 caveat)
- CLI: `droidbridge whatsapp scan` , `droidbridge whatsapp analyze --cutoff`

3.2 Backup Sub-Phase

- Full and selective backup using Phase 2 transfer engine
- Direct-to-external-drive support
- Post-backup verification (count + size match)
- Backup report + session log generation
- CLI: `droidbridge whatsapp backup --dest --type`
- Restore (media-only): push a backup folder back onto the phone's WhatsApp media path using the Phase 2 transfer engine (push mode), with the same conflict resolution and post-restore verification (see §4.9). Database/account restore is deferred — see 3.5.
- CLI: `droidbridge whatsapp restore --src --dest-path`
- Status Saver: pull currently-active `.Statuses` media (images/videos) for WhatsApp and WhatsApp Business into separate per-app subfolders (see §4.10)
- CLI: `droidbridge whatsapp save-status --dest`

3.3 Organization Sub-Phase (Optional Step)

- Organize-by-date for Voice Notes, Images, Video (Received/Sent/Private)
- Organize-by-category for Documents (PDFs, Archives, Code, Engineering, etc.)
- MIME-type extension fixing for files with no/wrong extension
- Double-dot filename fixing
- Bad year folder detection and correction (student IDs, serial numbers)
- CLI: `droidbridge whatsapp organize --src --type`

3.4 Cleanup Sub-Phase

- Pre-deletion preview report (count + size by year/type)
- Keep/delete rules by file type and date cutoff
- Mandatory typed confirmation (`'YES DELETE'`)
- Batch deletion (500 files per ADB call)
- Resume interrupted deletion via rescan
- Post-deletion verification and report
- Block deletion if backup not verified
- CLI: `droidbridge whatsapp delete --before --keep`

3.5 Database Sub-Phase

- Backup `Databases/` , `Backups/` , `accounts/` folders
- CLI: `droidbridge whatsapp backup-db --dest`
- Database restore is intentionally out of scope for the CLI toolkit: WhatsApp only picks up a restored `msgstore.db.crypt*` during a fresh install/login (not while already set up), and `Android/data/com.whatsapp` is restricted on Android 11+. Document this caveat rather than implementing an automated `restore-db` command.

Exit Criteria:

- Full WhatsApp scan on a real device with 500k+ files completes in under 5 minutes
- Backup, organize, and delete workflow tested end-to-end on a real device
- Deletion never proceeds without a verified backup and typed confirmation
- All session logs (scripts/reports/summary) generated correctly

Phase 4 — Storage, Apps & Backup Manager + Cross-Platform

Goal: round out remaining modules and add Windows/macOS platform support.

Deliverables:

- Module 5 — Storage Analyzer: full breakdown, top apps, large files, duplicates, suggestions
- Module 8 — App Manager: list, clear cache, uninstall, APK extraction, bloatware manager
- Module 6 — Backup Manager: profiles, incremental backup, history, restore
- Platform abstraction layer: `core/platform/` (`windows.py` , `linux.py` , `macos.py`)
- Bundle ADB binaries for Windows and macOS
- Sleep inhibitor for Windows (`SetThreadExecutionState`) and macOS (`caffeinate`)
- Test full CLI suite on Windows and macOS

Exit Criteria:

- All CLI commands work identically on Linux, Windows, and macOS
 - Storage analyzer and app manager tested on a real device
 - Backup profiles save/load correctly and incremental backup skips unchanged files
-

Phase 5 — Reports & Analysis Module

Goal: unify all report generation into a consistent, reusable module.

Deliverables:

- Module 9 — Reports: TXT, HTML, CSV, JSON generators for all report types
- Session log management: consistent structure across all modules
- Storage trend comparison across multiple sessions
- CLI: `droidbridge report generate --format`

Exit Criteria:

- Every module's output can be exported in all 4 formats
 - HTML reports render correctly in a browser with no external dependencies
-

Phase 6 — GUI & Packaging

Complete (v1.0.0)

Goal: wrap the fully-working CLI in a desktop GUI and produce distributable executables.

Deliverables (all shipped):

- PyQt6 GUI with sidebar navigation for all 9 modules (Ctrl+1–9)
- GUI calls into the same `core/` and `modules/` logic as the CLI — no duplicated logic
- Progress bars, confirmation dialogs, dark/light theme toggle
- Status Saver panel: thumbnail grid with ffmpeg video thumbnails, checkbox selection
- Export button on every results table (TXT/CSV/HTML/JSON)
- PyInstaller `--onedir` builds for Linux (`.deb` + tarball), Windows (zip), macOS (zip)
- ADB v37 official Google binaries bundled for all three platforms
- Packaging scripts: `scripts/package-linux.sh`, `package-windows.bat`, `package-macos.sh`
- Full documentation: `README.md`, `docs/USER_GUIDE.md`, `docs/INSTALL.md`

Exit Criteria (met):

- GUI performs every operation the CLI can perform
- Self-contained bundle runs on Linux without Python installed (Windows/macOS: builds prepared, real-device validation deferred)
- All destructive operations show confirmation dialogs in GUI

Recommended approach: complete Phases 1–3 fully and use the CLI for real backups before starting Phase 4. This validates the core WhatsApp toolkit against real-world data early.

17. Future Ideas / Research Backlog

Ideas raised during development that are **out of scope for the current 6-phase plan** but worth documenting for later research. Nothing here is scheduled — revisit only after Phase 6 is complete, and only with explicit user sign-off.

17.1 WhatsApp Cross-Device "Cloner" (like paid transfer tools)

Raised during the Phase 3.5 wrap-up: could DroidBridge clone WhatsApp chat history between phones the way paid tools (MobileTrans, dr.fone, iCareFone, etc.) do — Android → Android and Android → iOS?

Android → Android (same account):

- Closest to in-scope — builds on `whatsapp backup-db` (Databases/Backups/accounts) and `whatsapp backup` (Media).
- **Blocker:** `msgstore.db.crypt14` / `.crypt15` is encrypted with a key in `/data/data/com.whatsapp/files/key` — app-private storage, root-only on Android 11+ (same restriction already documented in §4.8/§4.9 for database restore). Without that key, WhatsApp on the target device can't decrypt a restored database.
- WhatsApp's own **"Chat Transfer"** feature (Wi-Fi Direct/QR, both phones running WhatsApp, same network) already solves this via its own device-to-device protocol — not something an external ADB tool can drive without root or the app's cooperation.

Android → iOS:

- This is what the paid tools actually do: reverse-engineer WhatsApp iOS's `ChatStorage.sqlite` (Core Data) schema, convert the Android `msgstore.db` (SQLite) + media into it, package the result as an iTunes/Finder-style backup, and restore it to the iPhone via `libimobiledevice` / `pymobiledevice3` (requires USB pairing trust with the iPhone).
- This is a **separate iOS-communication subsystem** — not ADB, not Android. The iOS schema changes with every WhatsApp release, which is why these tools are paid/subscription products with constant maintenance, and building/distributing one carries real reverse-engineering/ToS exposure.

Conclusion: out of scope for DroidBridge as designed (ADB-only, fully offline, source-available, Android-focused CLI/GUI). If ever pursued, it would be a **separate companion project**, not a DroidBridge module — DroidBridge's

existing `backup / backup-db / restore` commands already cover the Android-side data extraction such a project would need as input.

Open research questions for later (not scheduled):

1. Does WhatsApp's local-backup restore flow accept a `Databases/ + Media/` drop-in for a *fresh install, same phone number* without the original key — or does local (non-Drive) restore require key continuity regardless? (Needs real-device testing; unconfirmed either way.)
2. On a **rooted** source and target, is copying `/data/data/com.whatsapp/files/key` alongside `msgstore.db.crypt14` sufficient for cross-device restore? (Android → Android only.)
3. iOS side: survey `pymobiledevice3`'s backup create/restore support without iTunes/Finder, and document a current WhatsApp-iOS `ChatStorage.sqlite` schema as a starting point, if a separate project is ever started.

19. Version 1.0.0 — Implementation Notes & Deviations

This section records how the shipped v1.0.0 implementation differs from or extends the original spec above. The spec remains the canonical design reference; this section supplements it with ground-truth implementation details.

19.1 Phase Completion Summary

Phase	Status	Test count	Real-device validated
1 — Foundation	Complete	—	Realme RMX1931
2 — File Operations	Complete	—	Realme RMX1931
3 — WhatsApp Toolkit	Complete	—	Realme RMX1931
4 — Storage, Apps, Backup	Complete	—	Realme RMX1931
5 — Reports	Complete	—	Realme RMX1931
6 — GUI & Packaging	Complete	1647/1647	Realme RMX1931

19.2 ADB Binary

The bundled Linux ADB was initially from the Debian `android-sdk-platform-tools` package (v28.0.2). For the v1.0.0 release it was replaced with the official Google platform-tools v37.0.0-14910828 binary for all three platforms:

Platform	Path	Version
Linux	<code>droidbridge/resources/platform-tools-linux/adb</code>	v37.0.0

Platform	Path	Version
Windows	<code>droidbridge/resources/platform-tools-windows/adb.exe</code>	v37.0.0
macOS	<code>droidbridge/resources/platform-tools-darwin/adb</code>	v37.0.0

19.3 GUI Sub-Phases (Phase 6 breakdown)

The GUI was built in 6 sub-phases beyond what §16 described:

Sub-phase	Deliverable
6.1	GUI shell + Device page (sidebar, top bar, log panel, dark/light theme)
6.2	Files, Transfer, Search pages + real-device validation
6.3	WhatsApp Toolkit GUI (8 tabs) + real-device validation
6.4	Backup Manager GUI (6 tabs, including Contacts/Call Log export)
6.5	Storage Analyzer (5 tabs), App Manager (6 tabs), Reports (13 types)
6.6	Export buttons on all result tables; Linux/Windows/macOS packaging

19.4 Features Added Beyond Spec

- **Export button on every results table** (§14 only specified this for Search): all module pages now have an "Export..." button producing TXT/CSV/HTML/JSON from whatever is on screen.
- **Display name resolution for apps** via `aapt2 dump badging` with an on-disk cache at `~/.local/share/droidbridge/label_cache/<serial>.json`, resolved in background threads.
- **WhatsApp Status Saver video thumbnails**: when `ffmpeg` is on PATH, the first frame of each `.mp4` / `.3gp` /etc. status video is extracted as a JPEG thumbnail for the grid.
- **Contacts & Call Log export** added to Backup Manager (§6.6): exports to vCard/CSV/JSON via `adb shell content query`, backup-only (no restore).
- **Glob/Regex toggle** on the Search name field, plus a MIME-type dropdown filter with mutual-exclusivity guard.
- **New Folder** button on the Transfer page remote-path field: creates a subfolder on the device via `adb shell mkdir -p`.
- **Session logging framework**: all CLI and GUI operations write timestamped logs to `session_logs/` in the working directory.
- **pm list packages as primary app source**: the app listing uses `pm list packages` as the authoritative package list and merges storage sizes from `dumpsys diskstats` — newly installed apps appear immediately without the diskstats async delay.

19.5 Features Deferred from Spec

Several spec items were explicitly deferred after feasibility analysis. See `docs/DEFERRED_AND_FUTURE_WORK.md` for the full list with rationale. Key deferrals:

- **500-file batch pull/push** (§3.2): deletion uses 500-file batches; pull/push do per-file ADB calls to support per-file retry, progress signals, and cancel. A future refactor could add batching without losing those signals.
- **Transfer batch size display** (MB/s transfer speed) not shown in GUI (only CLI).
- **Storage trend comparison** (§9.2): the Reports page has a "Storage Trend" report type but it reports on the current session only; cross-session trend requires SQLite session history not yet implemented.
- **Mirror mode real-device validation** deferred (implemented in CLI, not validated live).
- **Windows/macOS real-device validation** deferred — CLI and GUI are cross-platform in design; platform-specific testing requires hardware not available in the development environment.

19.6 Known Limitations (non-bugs)

- App cache clear (`apps clear-cache`) targets the ADB USB connection; the `pm clear` ADB command clears both cache and data on some Android versions — users should clear individual app caches when data preservation matters.
- WhatsApp database restore is intentionally not implemented: `msgstore.db.crypt14` is encrypted with a key in app-private storage (`/data/data/com.whatsapp/files/key`) inaccessible without root on Android 11+.
- Large files (>2 GB) in a single ADB `pull` may encounter ADB timeout on slow connections — increase `AdbClient.DEFAULT_TIMEOUT` or use `--timeout` if building from source.

20. Deferred, Blocked & Future Work

All items that were evaluated and not shipped in v1.0.0 are documented in full detail in:

`docs/DEFERRED_AND_FUTURE_WORK.md`

That document is organized into:

1. **Deferred items** (12) — implemented partially or not at all, with specific reasons
2. **Blocked items** (3) — technically infeasible without root: full app data backup, WhatsApp orphaned-media detection, WhatsApp database restore
3. **Out-of-scope decisions** (5) — deliberate design decisions not to build certain features

4. **Platform & packaging gaps** (4) — Windows/macOS real-device validation, README finalization
5. **Future GUI enhancements** (6 ideas) — cancel-everywhere, label cache management, Status Saver select-all, scheduled backups, multi-device switcher
6. **Spec audit table** — every §1–§17 requirement mapped to / /

Priority future work (post v1.0.0)

The highest-value items to pursue after the current release:

1. **Cancel button on all long-running GUI operations** — currently only progress bars are shown; adding a Cancel signal to viewmodels would improve UX significantly.
2. **Per-file retry in GUI** — the transfer engine supports retry; wiring it to the GUI progress dialog would surface it to users.
3. **Storage trend across sessions** — requires persisting scan snapshots to SQLite; the `reports/` infrastructure is ready for it.
4. **Windows/macOS real-device validation** — the code is cross-platform; just needs hardware to test on.
5. **500-file batch pull/push** — decoupling progress signals from per-file granularity would unlock the §3.2 speed target.

21. Architecture Overview

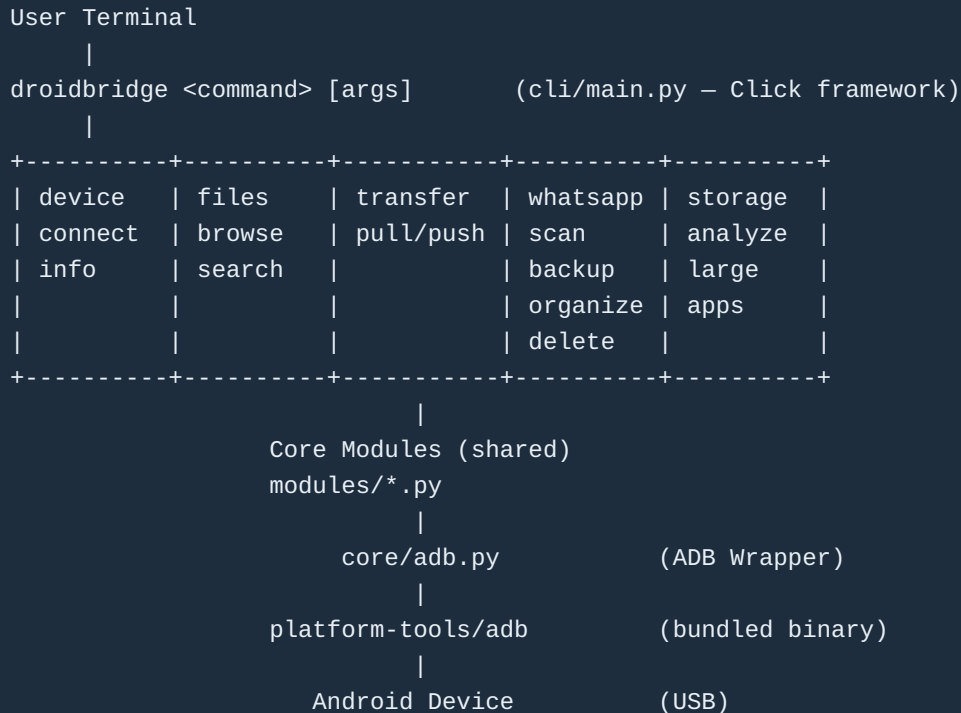
DroidBridge uses a **shared-core, dual-interface** architecture: both the CLI and the GUI call the same Python modules — no logic is duplicated. The only difference is the presentation layer.

21.1 Shared Core

```
droidbridge/  
  core/  
    adb.py          # ADB wrapper: devices(), shell(), pull(), push()  
    platform/       # OS-specific sleep inhibitor, path helpers  
  modules/  
    device.py       # Module 1 – Device Manager  
    files.py        # Module 2 – File Browser  
    transfer.py     # Module 3 – Smart Transfer Engine  
    whatsapp.py     # Module 4 – WhatsApp Toolkit  
    storage.py      # Module 5 – Storage Analyzer  
    backup.py       # Module 6 – Backup Manager  
    search.py       # Module 7 – Search & Discovery  
    apps.py         # Module 8 – App Manager  
    reports.py      # Module 9 – Reports & Analysis
```

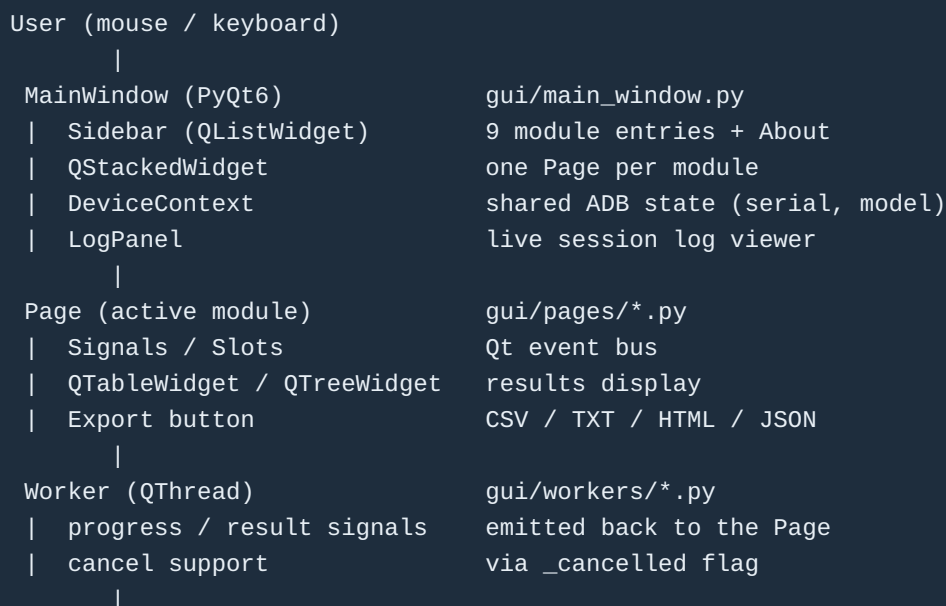
```
reports/      # TXT / HTML / CSV / JSON generators
utils/        # Size formatting, date parsing, helpers
```

21.2 CLI Architecture



Data flow: the CLI handler validates user input, calls the relevant module function, and prints the returned data structure. All output goes to stdout; session logs go to `session_logs/`. No GUI state is touched.

21.3 GUI Architecture



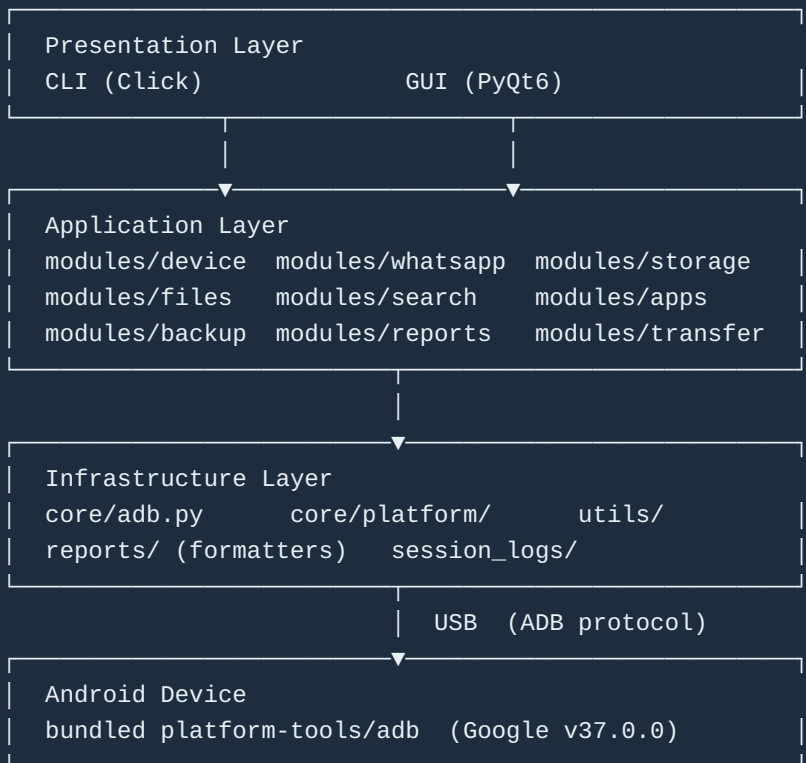
```

Core Modules (shared)      modules/*.py
    |
core/adb.py                (ADB Wrapper – same as CLI)
    |
Android Device (USB)

```

Key design rule: workers and pages never call `adb` directly — they always go through `core/adb.py`. This means every operation is automatically testable from the CLI without running the GUI.

21.4 Layered Component Map



22. About the Developer

DroidBridge was designed and built by:

Field	Details
Name	Ali Elmansoury
Title	Junior Embedded Software / Android Automotive Engineer
Email	ali.elmansoury21@gmail.com

Field	Details
GitHub	https://github.com/Ali-Elmansoury
LinkedIn	https://www.linkedin.com/in/ali-elmansoury/

DroidBridge grew out of a personal frustration with the painfully slow MTP transfers and the lack of good offline tools for managing large WhatsApp media archives on Android devices. Every feature in the tool was driven by real-world usage on real data.

DroidBridge is released under a proprietary source-available license — free for personal use, with a commercial license required for business use. Source code is publicly visible on GitHub.

Contributions, issues, and feedback are welcome.